

MODUL

PEMROGRAMAN BERORIENTASI OBJEK



Disusun oleh :

Tim Penyusun Modul

SEKOLAH TINGGI MANAJEMEN INFORMATIKA DAN KOMPUTER

AKAKOM

YOGYAKARTA

2020

KATA PENGANTAR

Puji Syukur kepada Allah SWT karena modul Pemrograman Berorientasi Obyek ini dapat terselesaikan. Modul ini dibuat karena tuntutan kurikulum baru atau disebut sebagai kurikulum 2019 STMIK AKAKOM yang mulai diberlakukan mulai tahun akademik 2019/2020.

Modul ini membahas tentang Kelas, Obyek, atribut, method, konstruktor, enkapsulasi dan komposisi, pewarisan, package, polimorfisme, kelas abstrak dan interface, exception handling dan tread.

Tentu saja dengan segala keterbatasan yang ada, modul ini jauh dari sempurna. Untuk itu masukan dan saran untuk perkembangan modul ini sangat diharapkan.

Akhirnya semoga modul ini berguna dalam membantu mahasiswa untuk mempelajari tentang Pemrograman Berorientasi Obyek.

Yogyakarta, Agustus 2019.

Penulis

DAFTAR ISI

KATA PENGANTAR	ii
DAFTAR ISI	iii
MODUL 1 KELAS dan OBJEK	1
MODUL 2 Atribut	6
MODUL 3 METHOD	11
MODUL 4 Konstruktor	16
MODUL 5 Package, Acces Modifier, Static.....	20
MODUL 6 ENKAPSULASI DAN KOMPOSISI	27
MODUL 7 PEWARISAN	32
MODUL 8 PEWARISAN	32
MODUL 9 KELAS ABSTAK	40
MODUL 10 INTERFACE.....	45
MODUL 11 POLIMORFISME.....	47
MODUL 12 EXCEPTION HANDLING	55
MODUL 13 INNER CLASS.....	61
MODUL 14 KONKURENSI	67

MODUL 1

KELAS dan OBJEK



CAPAIAN PEMBELAJARAN

Dapat membaca class diagram sederhana, Dapat menyebutkan bagian-bagian kelas, Dapat menjelaskan sintak kelas, Dapat membuat kelas sederhana, Dapat membedakan antara kelas dan objek, Dapat menciptakan Objek



KEBUTUHAN ALAT/BAHAN/SOFTWARE

1. Textpad sebagai teks editor / netbeans
2. JDK sebagai kompiler java



DASAR TEORI

Sebuah sistem yang dibangun berdasarkan metoda berorientasi objek adalah sebuah sistem yang komponennya di enkapsulasi menjadi kelompok data dan fungsi, yang dapat mewarisi atribut dan sifat dari komponen lainnya, dan komponen-komponen tersebut saling berinteraksi satu sama lain.

Bahasa Pemrograman yang berorientasi OBJEK memiliki kemampuan dalam pengelolaan program yang lebih diarahkan pada pembentukan objek. Dengan menerapkan konsep ini program akan lebih mudah untuk dikembangkan karena sifatnya yang lebih modular.

Dalam konsep object oriented akan kita temukan kata object dan class, class merupakan pola / template yang menggambarkan kumpulan object yang mempunyai sifat yang sama, perilaku, atau disebut dengan himpunan object sejenis. Sementara object adalah implementasi dari class. Adapun proses pembuatan obyek dari kelas disebut instantiasi atau implementasi dari kelas.

Tabel 1 adalah contoh ilustrasi kelas dan objek.

Tabel 1 Contoh class Mobil dan object-object nya

Kelas Mobil		Objek Mobil A	Objek Mobil B
Instan Variabel	nomor Plat	AB 3313 SY	AB 1234 AT
	Warna	Biru	Merah
	Manufaktur	Mitsubishi	Toyota
	Kecepatan	50 km/h	100 km/h
Instan Metode	method akselerasi		
	method belok		
	method rem		

Membuat Class dan Obyek dalam Java

❑ Elemen-elemen dasar dalam mendefinisikan kelas :

1. Field (variabel) : menyimpan data untuk setiap objek (implementasi dari atribut)
2. Constructor : setup objek di awal
3. Method : implementasi perilaku objek

```

class NamaKelas
{
    Fields
    Constructor
    Methods
}

```

Untuk membuat obyek digunakan kata kunci new :

NamaKelas NamaObyek = new NamaKelas();

Untuk memanggil method dapat digunakan sintaks sebagai berikut:

namaObyek.nama_method([parameter]);

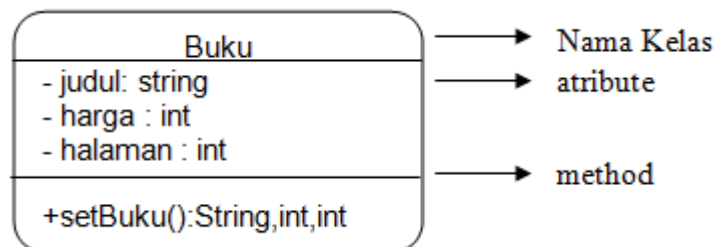
Terdapat 3 konsep penting dalam OOP, yaitu : **enkapsulasi**, **pewarisan**, dan **polymorfisme**. **Enkapsulasi** bertujuan untuk menyembunyikan kerumitan implementasi dan menyediakan interface bagi pengguna. Selain itu enkapsulasi juga berguna untuk proteksi data (read/write). **Pewarisan** adalah salah satu cara untuk menggunakan ulang kode-kode yang telah dibuat sebelumnya. Sedangkan **polymorfisme** adalah kemampuan obyek untuk bereaksi secara berbeda terhadap “pesan yang sama”, tergantung obyek yang menerima pesan tersebut. Kemampuan ini berkat adanya konsep method overriding dan method overloading.



PRAKTIK

Praktik 1 . Membuat kelas

Berikut ini adalah contoh class diagram untuk kelas Buku



Kode program:

```
class Buku
{
    String judul;
    int harga;
    int halaman;

    public void setBuku(String judul,int harga,int
halaman)
    {
        this.judul=judul;
        this.harga=harga;
        this.halaman=halaman;
    }
}
```

```
public void tampil()
{
    System.out.println("Judulbuku :"+judul);
    System.out.println("Harga buku : "+harga);
    System.out.println("Jumlah halaman : "+halaman);
}
}
```

Praktik 2. Membuat dan Menggunakan Obyek

Kelas Buku di atas hanya merupakan deklarasi dari obyek buku. Agar dapat digunakan maka Kelas Buku tersebut harus diinstantiasi. Dari satu buah kelas dapat dibuat banyak obyek. Obyek-obyek tersebut dapat dibedakan dengan suatu pengenalan yang berupa nama.

```
public class ObyekBuku
{
    public static void main(String[] args)
    {
        Buku buku1=new Buku();
        buku1.setBuku("PBO",75000,200);
        buku1.tampil();
    }
}
```



LATIHAN

1. Tambahkan 2 buah objek dari kelas Buku !
2. Panggil method setBuku() dan tampil()
3. Buatlah kelas Pegawai !
4. Buatlah 2 buah objek dari kelas Pegawai !



TUGAS

1. Implementasikan ke dalam program dari contoh Tabel 1 pada dasar teori !
2. Buatlah kelas handpone!
3. Buatlah 2 buah objek dari kelas handphone !



REFERENSI

Wagito, Sumiyatun, Fx. Henry Nugroho. 2015. Modul Praktikum Pemrograman Berorientasi Objek. STMIK AKAKOM. Yogyakarta

MODUL 2

ATRIBUT



CAPAIAN PEMBELAJARAN

1. Dapat mendefinisikan atribut kelas,
2. Dapat membuat dan memanggil atribut,
3. Dapat membedakan identifier dan konstanta,
4. Dapat mengidentifikasi atribut suatu kelas dan mengimplementasi menjadi kelas



KEBUTUHAN ALAT/BAHAN/SOFTWARE

1. Textpad / netbeans
2. JDK



DASAR TEORI

Variabel

Variabel adalah wadah yang menampung nilai-nilai yang digunakan dalam program Java. Setiap variabel harus dideklarasikan menggunakan tipe data. Misalnya, variabel dapat dinyatakan menggunakan salah satu dari delapan tipe data primitif: byte, short, int, long, float, double, char atau boolean. Dan, setiap variabel harus diberi nilai awal sebelum bisa digunakan.

contoh:

```
int myAge = 21;
```

Variabel "myAge" dinyatakan sebagai tipe int data dan diinisialisasi ke nilai 21.

Dalam definisi kelas, ada tiga jenis variabel.

- instance variables: Setiap metode dalam definisi kelas dapat mengakses variabel tersebut
- Parameter variabel: Hanya metode dimana parameter muncul dapat mengakses variabel tersebut. Ini adalah bagaimana informasi akan diteruskan ke objek.
- local variables: Hanya metode dimana parameter muncul dapat mengakses variabel ini lokal. Variabel ini digunakan untuk menyimpan hasil antara.

Konstanta

Sebuah konstanta adalah suatu tempat untuk menampung data yang nilainya selalu tetap dan tidak pernah berubah. Di java, kata kunci final dapat digunakan dengan tipe data primitif dan objek tetap (misalnya, String) untuk membuat konstanta.

Misal

```
final int DAYS_IN_JANUARY = 31;
```



PRAKTIK

Praktik 1: membuat kelas Lingkaran

```

public class Lingkaran
{
    final double phi = 3.14;
    double jari;

    public void setJari(double jari0)
    {
        this.jari=jari0;
    }

    public double jari()
    {
        return(jari);
    }

    public double luas()
    {
        double luas0;
        luas0=phi*jari*jari;
        return(luas0);
    }

    public void tampil()
    {
        System.out.println("jari jari: "+jari);
        System.out.println("luas: "+luas());
    }
}

```

Dari praktik 1 manakah yang merupakan atribut dan mana yang merupakan konstanta !

Praktik 2. Menggunakan kelas Lingkaran

```

public class Main1
{
    public static void main(String[] args)
    {
        Lingkaran a;
        a=new Lingkaran();
        a.setJari(10.00);
        a.tampil();
    }
}

```

Praktik 3. Menggunakan kelas Lingkaran

```

public class Main2
{
    public static void main(String[] args)
    {
        int i;
        Lingkaran[] a;
        a=new Lingkaran[5];

        for(i=0;i<5;i++)
        {
            a[i]=new Lingkaran();
        }
        a[0].setJari(00.00);
        a[1].setJari(10.00);
        a[2].setJari(20.00);
        a[3].setJari(30.00);
        a[4].setJari(40.00);
        for(i=0;i<5;i++)
        {
            System.out.println("Lingkaran ke: "+i);
            a[i].tampil();
        }
    }
}

```



LATIHAN

1. Modifikasi Kelas Buku pada modul 1 dengan menambahkan 2 atribut lagi !
2. Buat kelas mahasiswa dengan 5 atribut !



TUGAS

1. Jelaskan perbedaan antara variabel dan konstanta !
2. Buatlah kelas persegi panjang lengkap dengan atributnya untuk menentukan luas dan keliling persegi panjang !



REFERENSI

Wagito, Sumiyatun, Fx. Henry Nugroho. 2015. Modul Praktikum Pemrograman Berorientasi Objek. STMIK AKAKOM. Yogyakarta

MODUL 3 METHOD



CAPAIAN PEMBELAJARAN

Dapat membuat dan menggunakan berbagai method



KEBUTUHAN ALAT/BAHAN/SOFTWARE

1. Textpad
2. JDK



DASAR TEORI

Method

Sebuah method menjelaskan behaviour dari sebuah object. Method juga dikenal sebagai fungsi atau prosedur. Pendeklarasian method biasa dituliskan seperti berikut ini :

```
<modifier> <returnType> <name>(<parameter>*) {  
    <statement>*  
}
```

dimana,

<modifier> dapat menggunakan beberapa *modifier* yang berbeda

<returnType> dapat berupa seluruh tipe data, termasuk *void*

<name> *identifier* atas *class*

<parameter> ::= <type_parameter> <nama_parameter>[,]

Method Tanpa Return Value

Jenis method ini ditandai dengan *return type* yang berupa *void* dan pada bagian *statement* tidak terdapat keyword **return**. Pada pemrograman berorientasi obyek method jenis ini digunakan untuk membuat method *mutator*. Nama untuk method *mutator* sebaiknya diawali dengan kata set, misalnya: setName(), setAddress().

Method Dengan Return Value

Jenis method ini ditandai dengan *return type* selain *void* dan pada bagian *statement* terdapat keyword **return**. Pada pemrograman berorientasi obyek method jenis ini digunakan untuk membuat method *accessor*, dimana method *accessor* fungsinya adalah untuk membaca/mendapatkan nilai suatu atribut. Nama untuk method *accessor* sebaiknya diawali dengan kata get, misalnya: getName(), getAddress().

Method Overloading

Bahasa java mendukung method *overloading* , java dapat membedakan beberapa *method* dengan nama yang sama di dalam sebuah kelas namun parameternya berbeda. Hal ini sangat menguntungkan karena memudahkan kita dalam mengingat nama method, bayangkan bila program pada class Gambar harus diberi nama drawInteger(int i), drawString(String s), drawDouble(double d). Method *overloading* dibedakan oleh jumlah dan jenis tipe data parameternya.

Contoh method overloading :

```
public class Gambar{  
    public void draw(int i){  
        .....  
    }  
    public void draw(String s){  
        .....  
    }  
}
```

```

    }

    public void draw(double d){

        .....

    }

    public void draw(int i, double d){

        .....

    }

}

```



PRAKTIK

1. Praktik 1 (membuat method mutator dan method acesor)

```

public class Mahasiswa{
    private String nim,nama;

    //method mutator
    public void setNim(String nim) {
        this.nim=nim;
    }
    public void setName(String nama) {
        this.nama=nama;
    }

    //method acesor
    public String getNim() {
        return nim;
    }
    public String getName() {
        return nama;
    }

    public static void main(String args[]){
        Mahasiswa obj=new Mahasiswa();
        obj.setNim("175410001");
        obj.setName("Azkiya");
    }
}

```



```

        System.out.println("==Data==");
        System.out.println("Nim   : " + obj.getNim());
        System.out.println("Nama  : " + obj.getNama());
    }
}

```

2. Praktik 2 (Memuat method overloading)

```

class Hitung{
    private int a,b,c,d;

    void tambah(int a,int b){
        System.out.println(a+b);
    }
    void tambah(int a,int b,int c){
        System.out.println(a+b+c);
    }
    void tambah(int a,int b,int c, int d){
        System.out.println(a+b+c+d);
    }

    public static void main(String args[]){
        Hitung obj=new Hitung();
        obj.tambah(10,10,10);
        obj.tambah(20,20);
        obj.tambah(20,5,10,20);
    }
}

```

3. Praktik membuat method overloading untuk dengan perbedaan tipe parameter

```

class Calculation2{
    private int a,b;
    private double c,d;

    void sum(int a,int b) {
        System.out.println(a+b);
    }
    void sum(double c,double d) {
        System.out.println(c+d);
    }

    public static void main(String args[]){
        Calculation2 obj=new Calculation2();
        obj.sum(10.5,10.5);
        obj.sum(20,20);
    }
}

```

```
}
```



LATIHAN

1. Modifikasi praktik 3 dengan menambahkan method sum dengan tipe parameter yang berbeda selanjutnya panggil melali main program !
2. Modifikasi praktik 1 dengan menambahkan method TampilkanData yang digunakan untuk menampilkan data nim dan nama !
3. Buatlah kelas lengkap dengan atribut dan oprasi untuk menentukan luas seitiga !



TUGAS

1. Buatlah program lengkap dengan atribut dan oprasi yang digunakan untuk meghitung nilai pangkat dari suatu bilangan !
2. Soal dar dosen pengampu



REFERENSI

Wagito, Sumiyatun, Fx. Henry Nugroho. 2015. Modul Praktikum Pemrograman Berorientasi Objek. STMIK AKAKOM. Yogyakarta

MODUL 4

Konstruktor



CAPAIAN PEMBELAJARAN

Dapat membuat dan menggunakan konstruktor



KEBUTUHAN ALAT/BAHAN/SOFTWARE

1. Textpad
2. JDK



DASAR TEORI

Konstruktor adalah sebuah tipe khusus dari method yang digunakan untuk membuat dan menginisialisasi sebuah object baru.

Berikut ini adalah *property* dari konstruktor :

1. konstruktor memiliki nama yang sama dengan class
2. konstruktor tidak memiliki return value
3. konstruktor tidak dapat dipanggil secara langsung, namun harus dipanggil dengan menggunakan operator **new** pada pembentukan sebuah *class*.

Untuk mendeklarasikan konstruktor, kita tulis

```
<modifier> <className> (<parameter>) {  
  
    <statement>
```

}

Perbedaan method biasa dengan konstruktor adalah bahwa konstruktor harus memiliki nama yang sama dengan nama classnya dan tidak memiliki nilai kembalian (tipe-data).

Konstruktor dijalankan pada saat sebuah object diinisialisasi (menggunakan kata new).

Konstruktor default

Apabila anda tidak mendeklarasikan satu pun konstruktor, maka Java secara otomatis menambahkan konstruktor default ke dalam class yang kita buat walaupun tidak kelihatan pada kode program. Apabila kita mendeklarasikan satu atau lebih konstruktor maka java tidak akan menambahkan konstruktor default.

Konstruktor overloading

Pada konstruktor juga berlaku overloading, artinya boleh mendeklarasikan lebih dari satu konstruktor, asalkan memiliki parameter yang berbeda – beda.

Pada konstruktor :

1. Memiliki konsep yang sama dengan overloading method.
2. Dibedakan berdasarkan paramater (jumlah atau tipe data).

Pemanggilan konstruktor tergantung pada instansiasi objek.



PRAKTIK

1. Praktik 1 Membuat konstruktor

```
public class Titik{  
    int x;  
    int y;  
    public Titik(){  
        System.out.println("Konstruktor titik dijalankan!");  
    }  
}
```

2. Praktik 2 Membuat obyek dan pemanggilan konstruktor

```
public class TesTitik {  
    public static void main(String[] args) {  
        Titik a=new Titik();  
    }  
}
```

3. Praktik 3 Membuat konstruktor overloading dan pemanggilan konstruktor

Tambahkan konstruktor baru pada class Titik

```
public Titik(int x, int y) {  
    this.x = x;  
    this.y = y;  
    System.out.println("Konstruktor titik 2  
dijalankan!");  
}
```

Tambahkan skrip berikut pada class TesTitik

```
Titik b=new Titik(10,10);
```

4. Praktik 4 Overloading konstruktor

```
public class Pegawai  
{  
    String NamaPegawai;  
    int IdPegawai;  
    String PosisiPegawai;  
  
    Pegawai (String nama,int Id, String posisi) // konstruktor  
    {  
        NamaPegawai = nama;  
        IdPegawai = Id;  
        PosisiPegawai = posisi;  
    }  
    Pegawai () //konstruktor  
    {  
        NamaPegawai = "Azkiya";  
        IdPegawai = 2514;  
        PosisiPegawai = "Staf pengajar";  
    }  
    void Show()  
    {  
        System.out.println("Informasi Pegawai");  
        System.out.println("Nama      : "+NamaPegawai);  
        System.out.println("Id      : "+IdPegawai);  
        System.out.println("Posisi   : "+PosisiPegawai);  
    }  
}
```

```
public static void main(String args[])
{
    Pegawai pegawai1 = new Pegawai();
    Pegawai pegawai2 = new Pegawai("Zahwa",3313,"Staf
Akademik");
    pegawai1.Show();
    pegawai2.Show();
}
```



LATIHAN

1. Modifikasi class buku pada modul 1 dengan menggunakan konstruktor !



TUGAS

1. Buatlah class Komputer lengkap dengan atribut, method dan konstruktor !



REFERENSI

Wagito, Sumiyatun, Fx. Henry Nugroho. 2015. Modul Praktikum Pemrograman Berorientasi Objek. STMIK AKAKOM. Yogyakarta

MODUL 5

Package, Acces Modifier, Static



CAPAIAN PEMBELAJARAN

Dapat menggunakan paket java,
Dapat membuat dan menggunakan package yang dibuat,
Dapat mengatur hak akses member



KEBUTUHAN ALAT/BAHAN/SOFTWARE

1. Textpd / netbeans
2. JDK



DASAR TEORI

Seperti yang diketahui, Java adalah pemrograman dengan menggunakan banyak kelas. Kelas-kelas tersebut dikelompokkan ke dalam kategori tertentu yang berhubungan disebut dengan *packages* (paket) . Misalnya saja paket javax.swing.* berarti semua kelas yang berhubungan dengan javax swing berada dalam direktori tersebut.

Langkah-langkah untuk membuat kelas yang *reusable* adalah sebagai berikut:

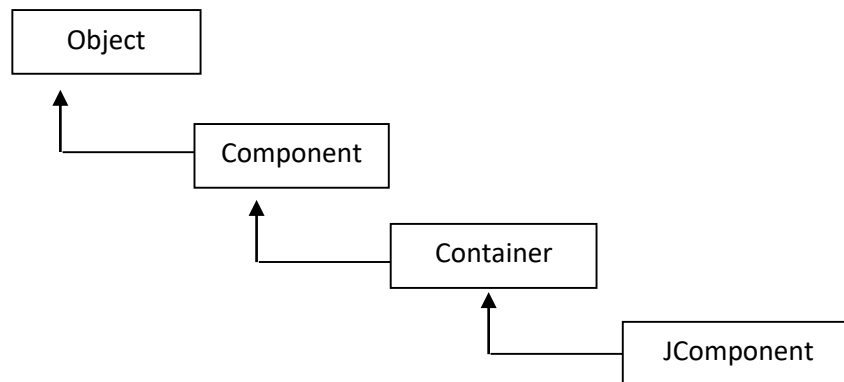
1. Deklarasikan *public class*. Jika kelas tidak bersifat public, dia hanya bisa dipergunakan oleh kelas lain di dalam paket yang sama.

2. Pilih nama paket, dan tambahkan sebuah *package declaration* ke file *source code* untuk deklarasi kelas *reusable*. Hanya bisa ada satu deklarasi paket saja di dalam sebuah file *source code* Java dan dia harus mendahului semua deklarasi dan pernyataan lain di dalam file tersebut.
3. Kompilasi kelas tersebut sehingga dia ditempatkan di dalam struktur direktori paket yang sesuai.
4. Import *reusable class* ke dalam sebuah program, dan gunakan kelas tersebut.

Fitur ini menyediakan mekanisme untuk mengatur class dan interface dalam jumlah banyak dan menghindari konflik pada penamaan.

Menggunakan Salah satu Paket Miliknya Java : Komponen Swing

Komponen	Deskripsi
JLabel	Area dimana teks yang tidak bisa diedit atau sebuah icon dapat ditampilkan
JTextField	Area dimana user menginputkan data dari keyboard. Area ini juga bisa menampilkan informasi
JButton	Area yang memicu sebuah event ketika diklik dengan mouse
JCheckBox	Komponen GUI yang bisa dipilih atau tidak dipilih
JComboBox	Sebuah drop-down list dari item dari mana user dapat membuat pilihan dengan meng-klik sebuah item dalam list atau mungkin dengan menuliskan dalam box
JList	Area yang berisi sebuah list dari item dari mana user dapat membuat pilihan dengan meng-klik pada beberapa elemen dalam list. Beberapa elemen dapat dipilih
JPanel	Sebuah kontainer dalam mana komponen dapat ditempatkan atau diorganisasi



Gambar 1. Superclass untuk banyak komponen Swing

Gambar 1 memperlihatkan sebuah hirarki inheritance yang berisi class yang mendeklarasikan atribut dan lingkungan yang berhubungan dengan banyak komponen Swing. Class object adalah superclass dari hirarki class Java. Class Component (package java.awt) adalah subclass dari Object, class Container (package java.awt) adalah subclass dari Component, dan class Jcomponent (package javax.swing) adalah subclass dari Container.

Struktur Package

```
package nm_package;
import registration.processing.*;
import java.util.List;
import java.lang.*;      //imported by default
class NmClass {
    /* details of NmClass */
}
```

Access Modifier

Public : Menyatakan bahwa kelas/method/attribute tersebut dapat diakses oleh kelas lain dimanapun.

Protected : Menyatakan bahwa kelas/method/attribute tersebut dapat diakses oleh kelas lain yang berada dalam satu package atau kelas lain tersebut merupakan turunannya.

Private : Menyatakan bahwa kelas tersebut tidak dapat diakses sama sekali oleh kelas lain bahkan juga tidak dapat diturunkan.



PRAKTIK

1. Menggunakan paket miliknya java. Cobalah program berikut yang menggunakan paket bawaannya java.

```
import java.util.Scanner;
public class Jumlah
{
    public static void main(String args[])
    {
        Scanner masuk = new Scanner(System.in);
        int gajiPokok,tunjangan, jumlahGaji;
        String nama;
        System.out.print("Nama : ");
        nama = masuk.nextLine();
        System.out.print("Gaji pokok : ");
        gajiPokok = masuk.nextInt();
        System.out.print("tunjangan : ");
        tunjangan = masuk.nextInt();
        jumlahGaji = gajiPokok + tunjangan;
        System.out.println("nama    = "+nama);
        System.out.println("Jumlah gaji = " + jumlahGaji);
    }
}
```

```
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;

public class LabelTest extends JFrame {
    private JLabel label;

    public LabelTest()
    {
        super( "Mencoba JLabel" );
        Container container = getContentPane();
        container.setLayout( new FlowLayout() );
        label = new JLabel( "Label dengan text" );
        label.setToolTipText( "Ini adalah label1" );
        container.add( label );
        setSize( 500, 400 );
    }
}
```

```

    setVisible( true );
}

public static void main( String args[] )
{
    LabelTest aplikasi = new LabelTest();
    aplikasi.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
}
}

```

2. Membuat sebuah paket sederhana

Tuliskan program berikut dalam folder mahasiswa yang berada di dalam direktori kerja anda. Jika belum ada buat dulu.

```

package mahasiswa;
public class paket
{
    public static void isiPaket(){
        System.out.println("ini adalah hasil import");
    }
}

```

Simpan dengan nama paket.java pada direktori mahasiswa.

3. Menggunakan paket hasil buatan sendiri. Cobalah program berikut. Simpan dalam direktori kerja anda (di luar folder mahasiswa).

```

import mahasiswa.*;
public class cobaPaket
{
    public static void main(String args[])
    {
        paket coba=new paket();
        coba.isiPaket();
    }
}

```

4. Modifikasi class paket menjadi seperti berikut :

```

package mahasiswa;
public class paket
{
    String nama;
    int umur;
    public static void isiPaket(){
        System.out.println("ini hasil import");
    }
    public void setData(int umur, String nama){
        this.umur = umur;
        this.nama = nama;
    }
}

```

```

    }
    public void info(){
        System.out.println("Nama : "+nama);
        System.out.println("Umur : "+umur);
    }
}

```

5. Panggil method tersebut dari class cobaPaket!



LATIHAN

2. Ketikkan program berikut

```

class Film
{
    private String judul;
    private String jenis;
    private int durasi;

    public void setFilm(String judul,String jenis,int durasi)
    {
        this.judul=judul;
        this.jenis=jenis;
        this.durasi=durasi;
    }
}

```

```

public class TesFilm
{
    public static void main(String[] args)
    {
        Film film1=new Film();
        film1.setFilm("Warkop DKI","Komedi",120);
        System.out.println("Judul film :"+film1.judul);
        System.out.println("jenis film : "+film1.jenis);
        System.out.println("dursi : "+film1.durasi);
    }
}

```

- Apa yang terjadi saat class TesFilm dikompile? Mengapa demikian?
- Ubah modifier atribut pada class Film menjadi public ! jalakan dan jelaskan hasilnya !



TUGAS

1. Buatlah program dengan menggunakan paket java yang lain !
2. Tugas diberikan oleh dosen pengampu di kelas !



REFERENSI

Wagito, Sumiyatun, Fx. Henry Nugroho. 2015. Modul Praktikum Pemrograman Berorientasi Objek. STMIK AKAKOM. Yogyakarta

MODUL 6

ENKAPSULASI DAN KOMPOSISI



CAPAIAN PEMBELAJARAN

Dapat mengimplementasi konsep encapsulasi pada aplikasi serta dapat membuat aplikasi yang berelasi komposisi



KEBUTUHAN ALAT/BAHAN/SOFTWARE

1. Textpad
2. JDK



DASAR TEORI

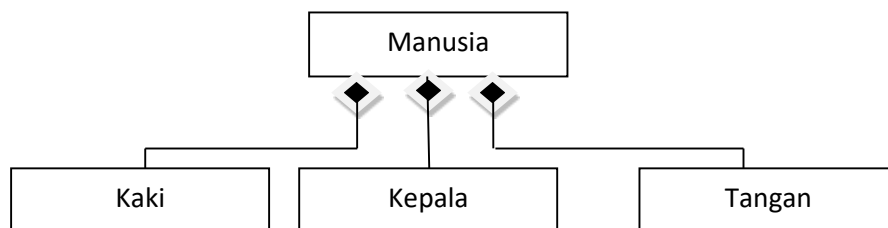
Enkapsulasi

Adalah salah satu konsep fundamental dalam *object oriented*, lainnya adalah pewarisan, polimorfisme dan abstrak. Enkapsulasi adalah suatu cara pengemasan data dan fungsi dalam sebuah kelas yang terlindungi dari akses secara sembarangan dari pihak luar. Oleh karena itu disebut sebagai proses pengkapsulan, dimana seperti obat bubuk yang terlindungi oleh bungkus kapsul supaya tidak mudah terkontaminasi bakteri. Teknik melakukan enkapsulasi adalah dengan memberikan hak akses private untuk atribut kelas, lalu untuk mengaksesnya dilakukan melalui method yang diberi hak akses public. Jika atribut dideklarasikan dengan hak akses private maka tidak dapat diakses dari luar kelas, maka data tersebut terlindungi. Keuntungan menggunakan enkapsulasi adalah

dapat memodifikasi suatu implementasi program tanpa perlu membongkar kode aslinya.

Komposisi

Dalam *object oriented*, satu kelas dapat berelasi dengan kelas lain untuk menggunakan fungsionalitas *method* yang disediakan oleh kelas lain. Hubungan relasi antar kelas diantaranya adalah komposisi. Dua buah kelas dikatakan memiliki relasi komposisi jika salah satu kelas bersifat mempunyai(*own*) kelas yang lain dan apabila pemilik kelas (*owner*) dihapuskan maka kelas yang lain tidak bisa berfungsi. Contohnya kelas manusia mempunyai kelas kepala, kelas tangan, kelas kaki. Bila kelas manusia tersebut dihapus maka kelas kepala, tangan, kaki tidak bisa berfungsi.



PRAKTIK

Praktik Enkapsulasi

1. Ketikkan kode program berikut ini

```
public class Pegawai {
    private String nama;
    private String jabatan;
    private int gaji;

    public String getJabatan() {
        return jabatan;
    }
}
```

```

    }
    public void setJabatan(String jabatan) {
        this.jabatan = jabatan;
    }
    public long getGaji() {
        return gaji;
    }
    public void setGaji(int gaji) {
        this.gaji= gaji;
    }
    public String getNama() {
        return nama;
    }
    public void setNama(String nama) {
        this.nama = nama;
    }

    public void cetakPegawai(){
        System.out.println("");
        System.out.println("Nama Pegawai:"+this.nama);
        System.out.println("Jabatan:"+this.jabatan);
        System.out.println("Gaji Pokok \t:"+this.gaji);
    }
}

```

2. Atribut nama,jabatan, gaji memiliki hak akses private untuk mempertahankan prinsip enkapsulasi. Mengakses atribut tersebut harus menggunakan method yang hak aksesnya bukan private.

```

public class TestEnkapsulasi{
    public static void main(String[] args) {
        Pegawai dataPeg=new Pegawai();
        dataPeg.setNama("Budi");
        dataPeg.setJabatan("Supervisor");
        dataPeg.setGaji(4000000);
        dataPeg.cetakPegawai();
    }
}

```

Praktik Komposisi

3. Ketikkan program berikut ini

```

public class Perusahaan {
    private String nmPerush;
    private String alamat;
    private Pegawai peg; //komposisi dengan relasi has-a
    public Perusahaan(){
        this.nmPerush="PT. Lancar Jaya";
        this.alamat="Yogyakarta";
        this.peg=new Pegawai();
    }
}

```



```

        peg.setNama("David");
        peg.setJabatan("Manager");
        peg.setGaji(5000000);
    }
    public void cetakPerush(){
        System.out.println("Nama Perusahaan:"+this.nmPerush);
        System.out.println("Alamat :"+this.alamat);
        peg.cetakPegawai();
    }
}

```

Program no.3 berelasi komposisi dengan program no.1

4. Ketikkan program berikut ini

```

public class TestKomposisi{
    public static void main(String[] args) {
        Perusahaan kantor1=new Perusahaan();
        kantor1.cetakPerush();
    }}

```



LATIHAN

1. Buatlah program Class SepedaMotor yang dilengkapi dengan 5 buah atribut (nama atribut bebas) dan method yang diperlukan, yang merupakan implementasi dari enkapsulasi !



TUGAS

1. Modifikasi kelas Buku pada modul 1 supaya program mengimplementasikan enkapsulasi !
2. Tugas diberikan oleh dosen pengampu !



REFERENSI

Wagito, Sumiyatun, Fx. Henry Nugroho. 2015. Modul Praktikum Pemrograman Berorientasi Objek. STMIK AKAKOM. Yogyakarta

MODUL 7 PEWARISAN



CAPAIAN PEMBELAJARAN

1. Dapat menggunakan overriding metode, menggunakan klausa super
2. Dapat menggunakan klausa final pada metode dan kelas



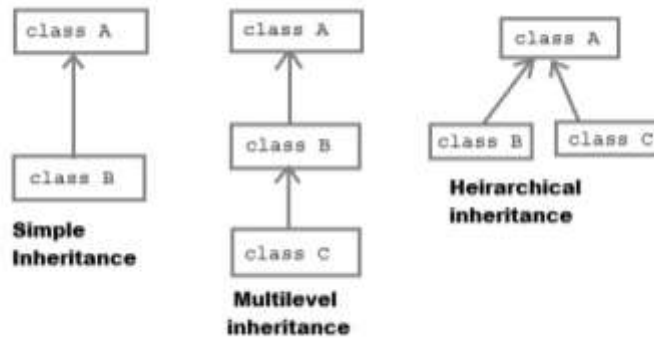
KEBUTUHAN ALAT/BAHAN/SOFTWARE

1. Textpad
2. JDK



DASAR TEORI

Pewarisan (Inheritance) merupakan sifat dalam bahasa berorientasi objek yang memungkinkan sifat-sifat dari suatu kelas diturunkan ke kelas lain. Sistem OOP memungkinkan kita untuk mendefinisikan suatu kelas baru dengan mewarisi sifat dari kelas lain yang sudah ada. Penurunan sifat ini bisa dilakukan dilakukan secara bertingkat-tingkat, sehingga semakin ke bawah maka kelas tersebut semakin spesifik, contoh, apabila kelas B adalah turunan dari kelas A, dalam proses pembuatan kelas turunan tersebut, Anda dapat menambahkan sifat dan perilaku baru ke dalam kelas B, yang sebelumnya tidak dimiliki dalam kelas A. Dalam terminologi java, kelas induk dinamakan dengan *superclass* dan kelas turunan dinamakan dengan *subclass*. untuk membuat kelas anak atau kelas turunan berdasarkan class yang ada, anda dapat menggunakan kata kunci *extends*. *Acces Modifier untuk mewujudkan turunan adalah protected*. *Berbagai macam bentuk hirarki pewarisan dapat dilihat pada gambar di bawah ini*.



PRAKTIK

1. Ketikkan kode program berikut ini untuk membuat kelas induk (*super class*)

```

class Manusia{
protected String nama;
protected int umur;

public Manusia(){
    this.nama="Sastro Wardoyo";
    this.umur=50;
}
public void cetak(){
    System.out.println("=====DATA PERSONAL=====");
    System.out.println("Nama:"+nama);
    System.out.println("Umur:"+umur);
}
}

```

2. Berikut ini adalah kode program untuk kelas turunannya (*sub class*). Kelas ini menerima warisan data nama dan umur dari kelas Manusia

```

class Mahasiswa extends Manusia{
private String nim;
private String jurusan;

public Mahasiswa(){

```

```

nama="Kurniawati";
umur=19;
this.nim="145419674";
this.jurusan="TI";
}

public void cetak(){
    System.out.println("=====DATA MAHASISWA=====");
    System.out.println("NIM :"+nim);
    System.out.println("Nama:"+nama);
    System.out.println("Umur:"+umur);
    System.out.println("Jurusan:"+jurusan);
}
}

//program utama
public class TestPewarisan{
public static void main(String args[]){
    Manusia orang1=new Manusia();
    orang1.cetak();
    System.out.println();
    System.out.println();

    Mahasiswa mhs1=new Mahasiswa();
    mhs1.cetak();
}
}

```



LATIHAN

1. Latihan diberikan oleh dosen pengampu pada saat praktikum.
2. Dikerjakan di laboratorium pada jam praktikum



TUGAS



REFERENSI

MODUL 8 PEWARISAN



CAPAIAN PEMBELAJARAN

1. Dapat mendefinisikan kelas turunan, Dapat membuat Objek turunan, Dapat mengakses atribut turunan
2. Dapat menerapkan kons-truktor pada sub kelas dalam pewarisan.



KEBUTUHAN ALAT/BAHAN/SOFTWARE

1. Textpad
2. JDK



DASAR TEORI

Dalam bab sebelumnya, kita berbicara tentang kelas-kelas yang super dan sub kelas. Jika kelas mewarisi metode dari kelas super, maka ada kesempatan untuk mengganti metode tersebut asalkan tidak ditandai final.

Manfaat utama adalah: kemampuan untuk menentukan perilaku yang khusus untuk jenis subclass yang berarti subclass dapat menerapkan metode kelas induk berdasarkan kebutuhannya.

Dalam hal berorientasi objek, utama berarti mengesampingkan fungsi dari metode yang ada



PRAKTIK

1. Membuat Hewan

```
class Hewan{

    public void move(){
        System.out.println("Hewans dapat move");
    }
}

class Kucing extends Hewan{
    public void move(){
        System.out.println("Kucings dapat walk dan run");
    }
}

public class TestKucing{
    public static void main(String args[]){
        Hewan a = new Hewan();
        Hewan b = new Kucing();
        a.move();
        b.move();
    }
}
```

2. Menggunakan kelas Hewan

```
class Hewan{
    public void move(){
        System.out.println("Hewans dapat move");
    }
}

class Kucing extends Hewan{
    public void move(){
        System.out.println("Kucings dapat walk dan run");
    }
    public void Meong(){
        System.out.println("Kucings dapat Meong");
    }
}

public class TestKucing{
    public static void main(String args[]){
        Hewan a = new Hewan();
        Hewan b = new Kucing();

        a.move();
        b.move();
    }
}
```



```
        b.Meong();  
    }  
}
```

Jalankan dan analisis hasilnya.

3. Menggunakan kata kunci super

Ketika memanggil versi superclass dari metode ditimpa kata kunci super digunakan

```
class Hewan{  
    public void move(){  
        System.out.println("Hewans dapat move");  
    }  
}  
  
class Kucing extends Hewan{  
    public void move(){  
        super.move();  
        System.out.println("Kucings dapat walk dan run");  
    }  
}  
  
public class TestKucing{  
    public static void main(String args[]){  
        Hewan b = new Kucing();  
        b.move();  
    }  
}
```



LATIHAN

1. Tuliskan soal latihan yang harus dikerjakan mahasiswa di sini (modul untuk mahasiswa tidak perlu diberi jawaban; modul untuk dosen perlu disertakan jawaban)
2.



TUGAS

1. Tuliskan soal tugas yang harus dikerjakan mahasiswa di sini (modul untuk mahasiswa tidak perlu diberi jawaban; modul untuk dosen perlu disertakan jawaban)
- 2.



REFERENSI

MODUL 9

KELAS ABSTAK



CAPAIAN PEMBELAJARAN

Dapat membuat dan menggunakan kelas abstrak



KEBUTUHAN ALAT/BAHAN/SOFTWARE

1. Textpad
2. JDK



DASAR TEORI

Abstraksi mengacu pada kemampuan untuk membuat kelas abstrak di OOP. Sebuah kelas abstrak adalah salah satu yang tidak bisa dipakai. Semua fungsi lain dari kelas masih ada, dan ladang-ladangnya, metode, dan konstruktor semua diakses dengan cara yang sama. Anda hanya tidak bisa membuat sebuah instance dari kelas abstrak.

Jika kelas abstrak dan tidak dapat dipakai, kelas tidak memiliki banyak digunakan kecuali subclass. Ini biasanya bagaimana kelas abstrak terjadi selama tahap desain. Sebuah kelas induk berisi fungsi umum dari kumpulan anak kelas, tapi kelas induk itu sendiri terlalu abstrak untuk digunakan sendiri.

Abstrak Kelas:

Gunakan kata kunci `abstract` untuk menyatakan `abstract class`. Kata kunci muncul dalam deklarasi kelas di suatu tempat sebelum kata kunci kelas



PRAKTIK

1. Membuat Karyawan

```
public abstract class Karyawan
{
    private String name;
    private String address;
    private int number;
    public Karyawan(String name, String address, int number)
    {
        System.out.println("Constructing Karyawan");
        this.name = name;
        this.address = address;
        this.number = number;
    }
    public double computePay()
    {
        System.out.println("Inside Karyawan computePay");
        return 0.0;
    }
    public void mailCheck()
    {
        System.out.println("Mailing a check to " + this.name
            + " " + this.address);
    }
    public String toString()
    {
        return name + " " + address + " " + number;
    }
    public String getName()
    {
        return name;
    }
    public String getAddress()
    {
        return address;
    }
    public void setAddress(String newAddress)
    {

```

```

        address = newAddress;
    }
    public int getNumber()
    {
        return number;
    }
}

```

Perhatikan bahwa tidak ada yang berbeda dalam kelas Karyawan ini. Kelas abstrak, tetapi masih memiliki tiga atribut, tujuh metode, dan satu konstruktor.

2. Membuat Kelas Karyawan

```

public class AbstractDemo
{
    public static void main(String [] args)
    {
        Karyawan e = new Karyawan("George W.", "Houston, TX", 43);

        System.out.println("\n Panggil mailCheck pakai Karyawan");
        e.mailCheck();
    }
}

```

Ketika dikompile akan didapatkan error sebagai berikut:

```

Karyawan.java:46: Karyawan is abstract; cannot be instantiated
        Karyawan e = new Karyawan("George W.", "Houston, TX", 43);
                        ^

```

1 error

3. Extend kelas Abstrak

```

public class Gaji extends Karyawan
{
    private double Gaji; //Annual Gaji
    public Gaji(String name, String address, int number, double
Gaji)
    {
        super(name, address, number);
        setGaji(Gaji);
    }
    public void mailCheck()
    {
        System.out.println("Within mailCheck Gaji");
    }
}

```

```

        System.out.println("Mailing check to " + getName()
+ " with Gaji " + Gaji);
    }
    public double getGaji()
    {
        return Gaji;
    }
    public void setGaji(double newGaji)
    {
        if(newGaji >= 0.0)
        {
            Gaji = newGaji;
        }
    }
    public double computePay()
    {
        System.out.println("Computing Gaji pay for " + getName());
        return Gaji/52;
    }
}

```

4. Menggunakan Gaji

Di sini, kita tidak bisa instantiate Karyawan baru, tetapi jika kita instantiate objek Gaji baru, objek Gaji akan mewarisi tiga bidang dan tujuh metode dari karyawan

```

public class AbstractDemo
{
    public static void main(String [] args)
    {
        Gaji s = new Gaji("Mohd Mohtashim", "Ambehta, UP", 3,
3600.00);
        Karyawan e = new Gaji("John Adams", "Boston, MA", 2,
2400.00);

        System.out.println("Panggil mailCheck pakai Gaji");
        s.mailCheck();

        System.out.println("\n Panggil mailCheck pakai Karyawan");
        e.mailCheck();
    }
}

```



LATIHAN

1. Tuliskan soal latihan yang harus dikerjakan mahasiswa di sini (modul untuk mahasiswa tidak perlu diberi jawaban; modul untuk dosen perlu disertakan jawaban)
2.



TUGAS

1. Tuliskan soal tugas yang harus dikerjakan mahasiswa di sini (modul untuk mahasiswa tidak perlu diberi jawaban; modul untuk dosen perlu disertakan jawaban)
- 2.



REFERENSI

MODUL 10 INTERFACE



CAPAIAN PEMBELAJARAN

Dapat membuat dan menggunakan interface pada aplikasi



KEBUTUHAN ALAT/BAHAN/SOFTWARE

1. Textpad
2. JDK



DASAR TEORI

1. Tuliskan Teori-teori yang mendukung disini
2.



PRAKTIK

5. Tuliskan praktik yang harus dikerjakan mahasiswa disini
6.



LATIHAN

3. Tuliskan soal latihan yang harus dikerjakan mahasiswa di sini (modul untuk mahasiswa tidak perlu diberi jawaban; modul untuk dosen perlu disertakan jawaban)
4.



TUGAS

3. Tuliskan soal tugas yang harus dikerjakan mahasiswa di sini (modul untuk mahasiswa tidak perlu diberi jawaban; modul untuk dosen perlu disertakan jawaban)
- 4.



REFERENSI

MODUL 11

POLIMORFISME



CAPAIAN PEMBELAJARAN

Dapat membuat aplikasi yang mengimplementasi compile-time (early binding) polymorphisme, dapat membuat aplikasi yang mengimplementasi runtime (late binding) polymorphisme



KEBUTUHAN ALAT/BAHAN/SOFTWARE

1. Textpad
2. JDK



DASAR TEORI

Polimorfisme adalah kemampuan sebuah variabel reference untuk merubah behavior sesuai dengan apa yang dimiliki object.

Polimorfisme membuat objek-objek yang berasal dari subclass yang berbeda, diperlakukan sebagai objek-objek dari satu superclass. Hal ini terjadi ketika memilih method yang sesuai untuk diimplementasikan ke objek tertentu berdasarkan pada subclass yang memiliki method bersangkutan.

Polimorfisme dapat diterjemahkan pula sebagai “sebuah method yang sama namanya (homonim) tetapi mempunyai tingkah laku yang berbeda”. Komputer

membedakan method berdasarkan signature method (saat compile) atau berdasarkan reference object (saat runtime).

Ada dua bentuk polimorfisme

1. Overloading (compile-time (early binding) polymorphisme)

Overloading adalah salah satu cara penerapan dalam konsep polimorfisme. Overload merupakan pendefinisian ulang suatu metode dalam class yang sama. Syarat overload yaitu metode dan tipe parameter harus berbeda dalam kelas yang sama.

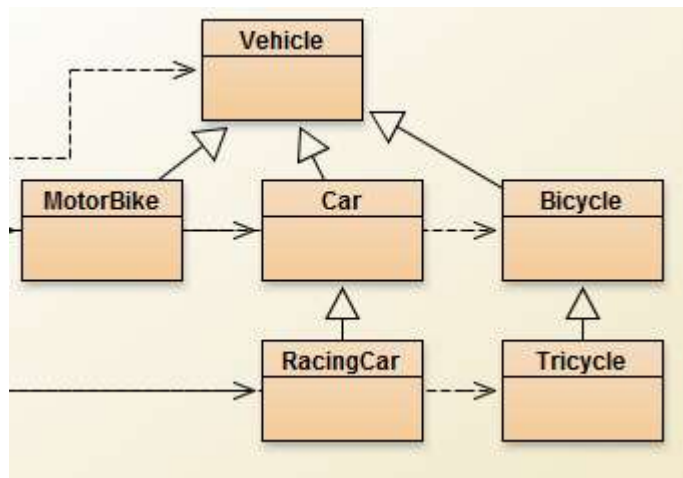
2. Overriding (runtime (late binding) polymorphisme)

Override merupakan pendefinisian ulang suatu metode oleh subclass. Syarat Override yaitu metode, return type, dan parameter harus sama. Jika tidak sama maka bukan dianggap sebagai override tetapi metode yang baru pada subclass.

Dikatakan **late-binding (run time) polymorphism** karena saat compile komputer tidak tahu method mana yang harus dipanggil, dan baru akan diketahui saat run-time. Run-time polymorphism dapat diterapkan melalui **overridden method**. Run time polymorphism juga disebut dengan istilah **dynamic binding**.

Anggaplah subclass meng-override method tertentu pada superclass. Andai kita cipta objek dari subclass dan memasukkannya dalam superclass. Walau objek subclass telah dimasukkan pada superclass, sewaktu objek tersebut memanggil method yang dioverride, ia tetap memanggil method yang versi subclassnya, bukan versi superclass.

Contoh



- Class Vehicle memiliki method move().
 - Class MotorBike meng-override method move().
- Java akan menunggu saat runtime untuk menentukan method move() yang mana akan dipanggil.



PRAKTIK

1. Praktik compile-time (early binding) polimorphisme dengan overloading method

```

class Jumlah{

    public int tambah(int x, int y){
        return x + y;
    }

    public int tambah(int x, int y, int z){
        return x + y + z;
    }

    public int tambah(double pi, int x){
        return (int)pi + x;
    }
}

public class Penjumlahan{

```

```

public static void main(String [] args)
{
    Jumlah obj = new Jumlah();
    System.out.println(obj.tambah(2,5)); // int, int
    System.out.println(obj.tambah(2, 5, 9)); // int, int, int
    System.out.println(obj.tambah(3.14159, 10)); // double, int
}
}

```

Pada praktik di atas polymorphism ditunjukkan dengan menggunakan berbagai method **tambah**. Komputer membedakan mana method yang dipanggil berdasarkan signature dari method yaitu parameter inputnya (jumlah parameter input, type data parameter input, dan urutan type data).

Bentuk polymorphism ini dinamakan **early-binding** (atau **compile-time polymorphism** karena komputer mengetahui mana method tambah yang dipanggil setelah mengcompile byte code . Jadi setelah proses compile ketika code sudah menjadi byte code, komputer akan “tahu” method mana yang akan dieksekusi Bentuk ini yang kita kenal dengan **overloaded method**.

2. Praktik compile-time (early binding) polimorphisme dengan overloading Konstruktor

```

class Penggajian{
    double gapok;
    double masa_kerja;
    Penggajian(double g, double mk)
    {
        gapok = g;
        masa_kerja=mk;
    }
    Penggajian()
    {
        gapok =0;
        masa_kerja=0;
    }
    Penggajian(double lembur)
    {
        gapok = masa_kerja = lembur;
    }
    double hitung_gaji()
    {
        return gapok*masa_kerja;
    }
}

```

```

    }

    class OverloadingKonstruktor
    {
        public static void main(String args[])
        {
            Penggajian Karyawan1 = new Penggajian(10,15);
            Penggajian Karyawan2 = new Penggajian();
            Penggajian Karyawan3 = new Penggajian(5);
            double gaji;
            gaji = Karyawan1.hitung_gaji();
            System.out.println("Gaji Karyawan 1= " +gaji);
            gaji = Karyawan2.hitung_gaji();
            System.out.println("Gaji Karyawan 2= " +gaji);
            gaji = Karyawan3.hitung_gaji();
            System.out.println("Gaji Karyawan 3= " +gaji);
        }
    }

```

3. Runtime polimorfisme dengan overriding method

```

class Kendaraan {
    public void info() {
        System.out.println("Info pada kendaraan : ");
    }
}

class Roda2 extends Kendaraan {
    public void info() {
        System.out.println ("Info pada Roda2");
    }
}

class Motor extends Roda2
{
    public void info() {
        System.out.println("Info pada Motor");
    }
}

public class Test {
    public static void main(String[] args){
        Roda2 roda2ku;
        Motor motorku;
        Kendaraan k = new Kendaraan();
        roda2ku=new Roda2();
        motorku=new Motor();
        k.info();
        k=roda2ku;
        k.info();
        k=motorku;
        k.info();
    }
}

```

4. Runtime polimorfisme dengan data member

```
class Induk {
    int x = 5;
    public void Info() {
        System.out.println("Ini class Induk");
    }
}
class Anak extends Induk {
    int x = 10;
    public void Info() {
        System.out.println("Ini class Anak");
    }
}
public class Test2 {
    public static void main(String args[])
    {
        Induk tes=new Anak();
        System.out.println("Nilai x = " + tes.x);
        tes.Info();
    }
}
```

Ketika program di atas dijalankan akan terlihat bahwa ketika diakses atribut x dari objek tes, maka yang muncul adalah nilai x dari super kelas, bukan atribut x dari subkelas. Akan tetapi ketika diakses method Info() dari objek tes, yang muncul adalah method Info() dari sub kelas, bukan method Info() dari superkelas

5. Runtime polimorfisme dengan passing parameter

```
class AlatGerak{
    void bergerak(){
        System.out.println(" Saya mampu bergerak");
    }
}
class Sayap extends AlatGerak{
    void bergerak(){
        System.out.println(" Saya bisa terbang");
    }
}
class Kaki extends AlatGerak{
    void bergerak(){
        System.out.println(" Saya bisa jalan-jalan");
    }
}
```

```

    }
}

class Burung {
    private AlatGerak alatGerak=new AlatGerak();
    Burung(){
        System.out.println("Hai saya Burung");
    }

    public void bergerak() {
        alatGerak.bergerak();
    }

    public void setAlatGerak(AlatGerak alatGerak) {
        this.alatGerak = alatGerak;
        System.out.println("Sekarang saya pakai " + alatGerak);
    }
}

class BurungTest{
    public static void main(String[] args){
        Burung merpati=new Burung();
        merpati.bergerak();
        Sayap sayap=new Sayap();
        Kaki kaki=new Kaki();
        merpati.setAlatGerak(sayap);
        merpati.bergerak();
        merpati.setAlatGerak(kaki);
        merpati.bergerak();
    }
}

```

Method `setAlatGerak` dapat menerima berbagai type yaitu Sayap dan Kaki. Hal ini legal karena pada kenyataannya Sayap dan Kaki adalah `AlatGerak`. Saat method `bergerak()` milik object `trutle` dipanggil, method yang dipanggil adalah sesuai dengan method yang dimiliki oleh type yang diberikan



LATIHAN

1. Tuliskan soal latihan yang harus dikerjakan mahasiswa di sini (modul untuk mahasiswa tidak perlu diberi jawaban; modul untuk dosen perlu disertakan jawaban)
2.



TUGAS

1. Tuliskan soal tugas yang harus dikerjakan mahasiswa di sini (modul untuk mahasiswa tidak perlu diberi jawaban; modul untuk dosen perlu disertakan jawaban)
- 2.



REFERENSI

MODUL 12

EXCEPTION HANDLING



CAPAIAN PEMBELAJARAN

Dapat menggunakan perintah yang menangani Exception pada program, menggunakan throws untuk melempar eksepsi dan membuat Exception sendiri



KEBUTUHAN ALAT/BAHAN/SOFTWARE

1. Textpad
2. JDK



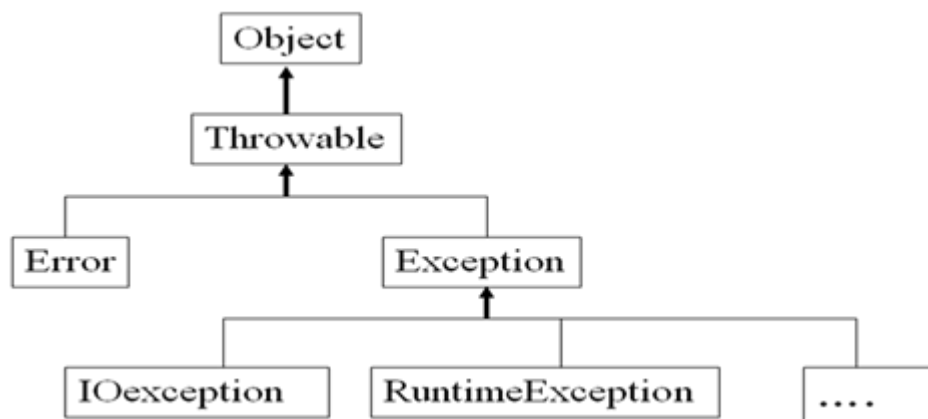
DASAR TEORI

Exception adalah sebuah masalah yang terjadi ketika program sudah pada tahap eksekusi, bukan saat tahap *compile*. Sebuah *exception* terjadi karena beberapa penyebab, misalnya: user memasukkan data yang tidak valid, file yang akan dibuka tidak ditemukan, koneksi yang belum tersambung atau tiba-tiba putus ditengah proses. Terdapat tiga macam exception:

1. Checked Exception : adalah exception yang biasanya terjadi karena kesalahan pengguna atau suatu masalah yang terjadi di luar perkiraan programmer. Contohnya sebuah file yang akan dibuka tapi file tersebut tidak dapat ditemukan maka exception terjadi. Kejadian seperti ini tidak terdeteksi pada saat tahap kompilasi.

2. Runtime Exception : adalah exception yang mungkin bisa dihindari oleh programmer. Kebalikan dari Checked Exception, bahwa Runtime Exception diabaikan pada tahap kompilasi.
3. Errors : adalah masalah yang terjadi diluar kontrol user ataupun programmer. Error sering diabaikan di dalam program karena anda tidak bisa menanganinya secara keseluruhan. Contohnya: jika kapasitas di memori penuh. Error seperti ini tidak terdeteksi pada tahap kompilasi

Hirarki kelas yang menangani exception:



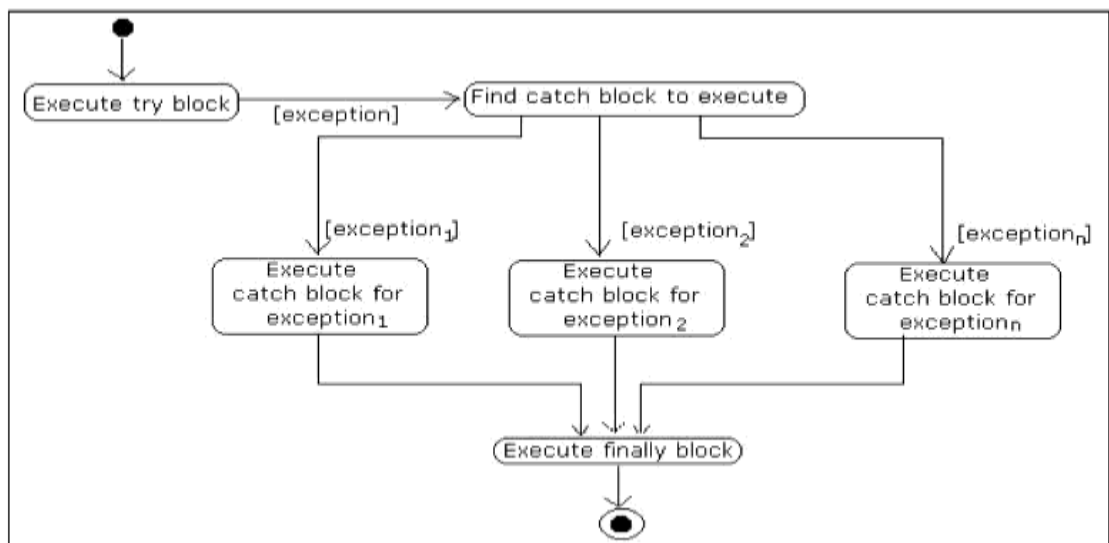
Struktur penulisan exception adalah :

```
try {  
    // blok program tempat menuliskan statement yang akan di uji  
    // apakah ada kesalahan atau tidak  
}  
catch ( <type exception1> <nama variabel>) {  
    // blok program tempat menuliskan aksi program bila sebuah  
    // exception terjadi  
}  
...  
catch ( <type exception2> <nama variabel>) {  
    // blok program tempat menuliskan aksi program bila sebuah  
    // exception terjadi  
}  
finally {  
    // blok program lanjutan  
}
```

Ketentuan dalam membuat exception adalah:

- Wajib membuat notasi blok
- Setiap blok try boleh memiliki lebih dari satu blok catch dan hanya boleh memiliki satu blok finally
- Blok catch dan blok finally harus muncul bersama blok try
- Blok try harus diikuti minimal satu blok catch, atau satu blok finally, atau kedua blok catch dan finally
- Setiap blok catch mendefinisikan penanganan exception. Di dalam header blok catch terdapat satu argumen yang akan ditangani oleh blok exception. Exception harus berasal dari class Throwable atau dari class turunannya.

Alur kerja dari exception :



PRAKTIK

- Membuat kelas tanpa exception

```

1. public class BagiNol {
2.     public static void main(String[] args) {
3.         System.out.println("Sebelum pembagian");
4.         System.out.println(5/0);
5.         System.out.println("Sesudah pembagian");
6.     }
7. }

```

Baris ke-5 tidak akan dieksekusi karena ada kesalahan pembagian dengan bilangan nol pada baris ke-4

2. Membuat kelas menggunakan exception

```

1. public class BagiNol2 {
2.     public static void main(String[] args) {
3.         System.out.println("Sebelum pembagian");
4.         try { System.out.println(5/0); }
5.         catch (Exception t) {
6.             System.out.print("Pesan kesalahan: ");
7.             System.out.println(t.getMessage());
8.         }
9.         System.out.println("Sesudah pembagian");
10.    }
11. }

```

Program tidak berhenti di baris ke -4 , dibuktikan dengan munculnya hasil dari baris ke-9

3. Menggunakan lebih dari satu statement catch

```

public class MultiCatchException{
public static void main(String args[]){
    try{
        System.out.println(5/0);
    }
    catch(ArithmeticException e){
        System.out.println(0);
    }
    catch(ArrayIndexOutOfBoundsException e){
        System.out.println(1);
    }
    catch(Exception e){
        System.out.println(2);
    }
}
}

```

Output adalah angka nol yang artinya pada blok try terdapat kesalahan *arithmetic* maka kesalahan tersebut ditangkap oleh blok catch dari kelas `ArithmeticException`

4. Menggunakan statement finally

```
public class MultiCatchException{
public static void main(String args[]){
    try{
        System.out.println(5/0);
    }
    catch(ArithmeticException e){
        System.out.println(0);
    }
    catch(ArrayIndexOutOfBoundsException e){
        System.out.println(1);
    }
    catch(Exception e){
        System.out.println(2);
    }
    finally{
        System.out.println(3);
    }
}
}
```

Bagian blok *finally* adalah bagian blok yang selalu dikerjakan. Output program adalah angka nol dan tiga, artinya setelah salah satu blok *catch* dieksekusi maka alur program selanjutnya adalah mengeksekusi bagian blok *finally*



LATIHAN

1. Tuliskan soal latihan yang harus dikerjakan mahasiswa di sini (modul untuk mahasiswa tidak perlu diberi jawaban; modul untuk dosen perlu disertakan jawaban)
2.



TUGAS

1. Tuliskan soal tugas yang harus dikerjakan mahasiswa di sini (modul untuk mahasiswa tidak perlu diberi jawaban; modul untuk dosen perlu disertakan jawaban)
- 2.



REFERENSI

MODUL 13

INNER CLASS



CAPAIAN PEMBELAJARAN

Dapat mendefinisikan inner class, nested class, non-static inner class, local class dan Anonymous class, serta dapat mengimplementasi konsep inner class pada aplikasi



KEBUTUHAN ALAT/BAHAN/SOFTWARE

1. Textpad
2. JDK



DASAR TEORI

Java mengijinkan kita mendefinisikan suatu kelas di dalam kelas lain, hal ini disebut sebagai *nested class*. Disebut nested karena class bersifat tersarang terhadap kelas – kelas utamanya, seperti halnya blok penyeleksian (if, for) yang tersarang pada blok penyeleksian lainnya atau method yang tersarang pada method lainnya. *nested class* dibagi menjadi dua kategori, yaitu *static* dan *non static nested class*. Istilah *inner class* biasa digunakan untuk *non static nested class*.

```
Class OuterClass{  
    ...  
    static class StaticNestedClass{
```



```

        ...
    }
    Class InnerClass{
        ...
    }
}

```

Nested class merupakan member dari kelas Outer(kelas terluar). Non static nested class (inner class) dapat mengakses semua member dari kelas Outer bahkan yang mempunyai hak akses private. Static nested class tidak mempunyai akses kepada member dari kelas Outer. Sebagai member dari Outer class, sebuah nested class dapat dideklarasikan dengan hak akses private, public, protected, sedangkan Outer class hanya bisa dideklarasikan dengan hak akses public.

Anonymous inner class adalah sebuah inner class yang dideklarasikan tanpa memberikan nama kelas. Anonymous class akan membuat kode program menjadi lebih ringkas karena dengan Anonymous class kita dapat mendeklarasikan dan meng-instant suatu kelas sekaligus dalam satu langkah



PRAKTIK

Membuat inner class

1. Tulislah program file **Luar.java** berikut:

```

public class Luar{
    private int angkaLuar;

    public Luar(int angkaLuar){
        this.angkaLuar=angkaLuar;
    }

    int getAngkaLuar(){
        return angkaLuar;
    }
    //inner class
        class Dalam{
            private int angkaDalam;

            public Dalam(){

```

```

        angkaDalam=9;
    }

    public void cetakDalam(){
        System.out.println("Ini angka luar:"+angkaLuar);
        System.out.println("Ini angka dalam:"+angkaDalam);
    }
} //batas inner class

public void cetakLuar(){
    Dalam dl=new Dalam();
    dl.cetakDalam();
}

public static void main(String args[]){
    Luar lu=new Luar(5);
    lu.cetakLuar();
}
} //oec out

```

2. Tulislah program file **RoundShape.java** berikut:

```

public abstract class RoundShape
{ // coordinates of center represented by an inner class
    protected class Center
    { int x,y; }
    protected Center C = new Center();
    protected float radiusOfCircle;

    // constructor
    public RoundShape(int xCenter, int yCenter, float radius)
    {
        C.x=xCenter;
        C.y=yCenter;
        radiusOfCircle = radius;
    }
    // abstract method
    abstract public float area();
}

```

3. Tulislah program file **Circle.java** berikut:

```

public class Circle extends RoundShape
{ // constructor
    public Circle(int xCenter, int yCenter, float radius)
    { super(xCenter, yCenter, radius); }

    public float area()
    {

```

```

float areaOfCircle =
(float) (Math.PI*Math.pow(radiusOfCircle,2.0));
return areaOfCircle;
}
}

```

4. Tuliskan program file **Sphere.java**

```

public class Sphere extends RoundShape
{

public Sphere(int xCenter, int yCenter, float radius)
{ super(xCenter, yCenter, radius); }

public float area()
{ float surfaceArea =
(float) (4.0*Math.PI*Math.pow(radiusOfCircle,2.0));
return surfaceArea;
}
}

```

5. Tuliskan **TestRoundShape.java**

```

import java.text.DecimalFormat;
public class TestRoundShape{
public static void main(String args[]){
DecimalFormat digit = new DecimalFormat ("0.##");
Circle lingk1=new Circle(20,5,10.0f);
System.out.println(digit.format(lingk1.area()));
}
}

```

Praktik membuat Anonymous Class

6. Tuliskan **TestAnonymous.java**

```

class Awal{
int x=8;
void methodAwal(){
System.out.println("Nilai x= "+x);
}
}

public class TestAnonymous{
public static void main(String args[]){
Awal noName=new Awal(){ //instant anonymous class
void methodAwal(){
x+=3;
System.out.println("x= "+x);
}
}
}

```

```
};  
noName.methodAwal();  
}  
}
```

Baris program di bawah tulisan "public static void main" adalah proses instant dari sebuah anonymous class. Obyek noName bukan obyek dari kelas Awal namun obyek dari anonymous class hal ini dapat dilihat dari nilai x=11, bukan x=8 ketika program dijalankan



LATIHAN

1. Tuliskan soal latihan yang harus dikerjakan mahasiswa di sini (modul untuk mahasiswa tidak perlu diberi jawaban; modul untuk dosen perlu disertakan jawaban)
2.



TUGAS

1. Tuliskan soal tugas yang harus dikerjakan mahasiswa di sini (modul untuk mahasiswa tidak perlu diberi jawaban; modul untuk dosen perlu disertakan jawaban)
- 2.



REFERENSI

MODUL 14 KONKURENSI



CAPAIAN PEMBELAJARAN

Dapat menggunakan kelas Threat untuk menangani konkurensi



KEBUTUHAN ALAT/BAHAN/SOFTWARE

1. Textpad
2. JDK



DASAR TEORI

3. Tuliskan Teori-teori yang mendukung disini
4.



PRAKTIK

1. Tuliskan praktik yang harus dikerjakan mahasiswa disini
2.



LATIHAN

1. Tuliskan soal latihan yang harus dikerjakan mahasiswa di sini (modul untuk mahasiswa tidak perlu diberi jawaban; modul untuk dosen perlu disertakan jawaban)
2.



TUGAS

1. Tuliskan soal tugas yang harus dikerjakan mahasiswa di sini (modul untuk mahasiswa tidak perlu diberi jawaban; modul untuk dosen perlu disertakan jawaban)
- 2.



REFERENSI
